Flutter Online Course

Section No.3

instructor

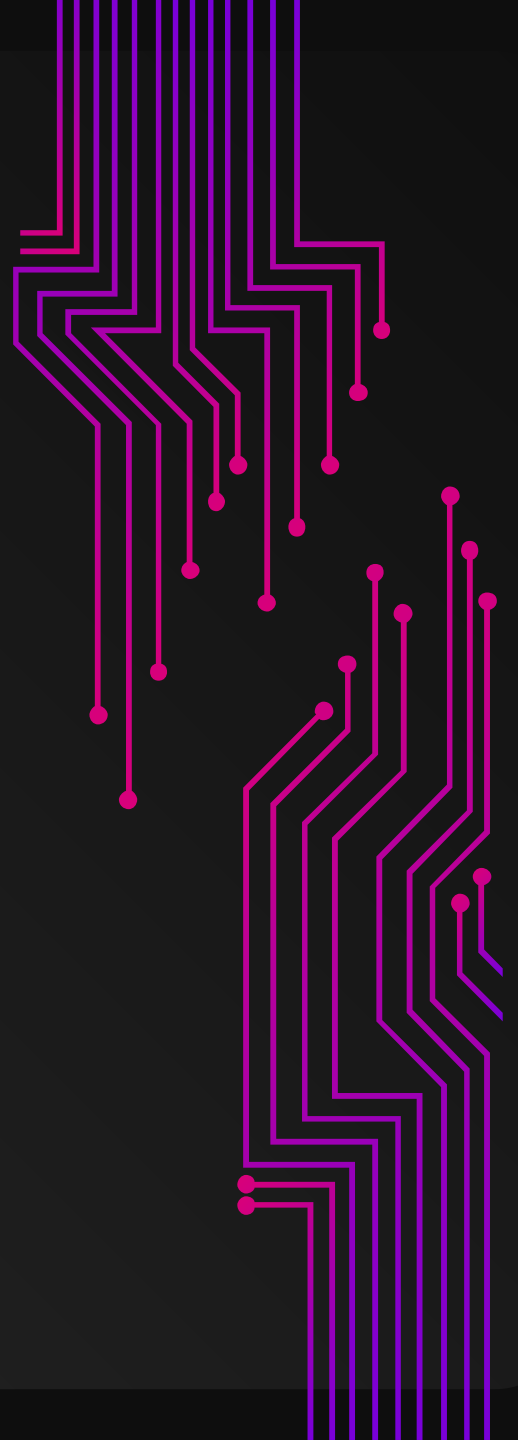
Youssef Hossam elden

Table of contents

01

Classes

Defining a Class
Constructor



Classes

what is the class ?

A class in Dart is a blueprint or template for creating objects. It defines the structure and behavior of objects of that class. A class can have properties (also called fields or instance variables) to store data and methods to perform actions or operations.

Defining a Class :

To define a class in Dart, use the class keyword followed by the class name. Here's an example of a simple class representing a Person...

Defining a Class :

```
class Person {
  String name;
  int age;

  // Constructor
  Person(this.name, this.age);

  // Method
  void greet() {
    print('Hello, my name is $name and I am $age years old.');
  }
}
```

Constructor

what is the Constructor ?

A constructor is a special method used to initialize the properties of an object when it is created. Dart provides a default constructor if one is not explicitly defined in the class. In the Person class example above, Person(this.name, this.age) is a constructor that takes two parameters and assigns them to the properties.

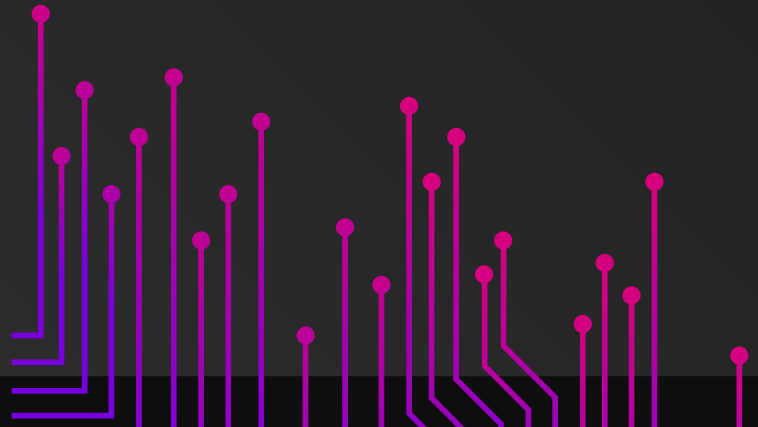
Defining a Constructor :

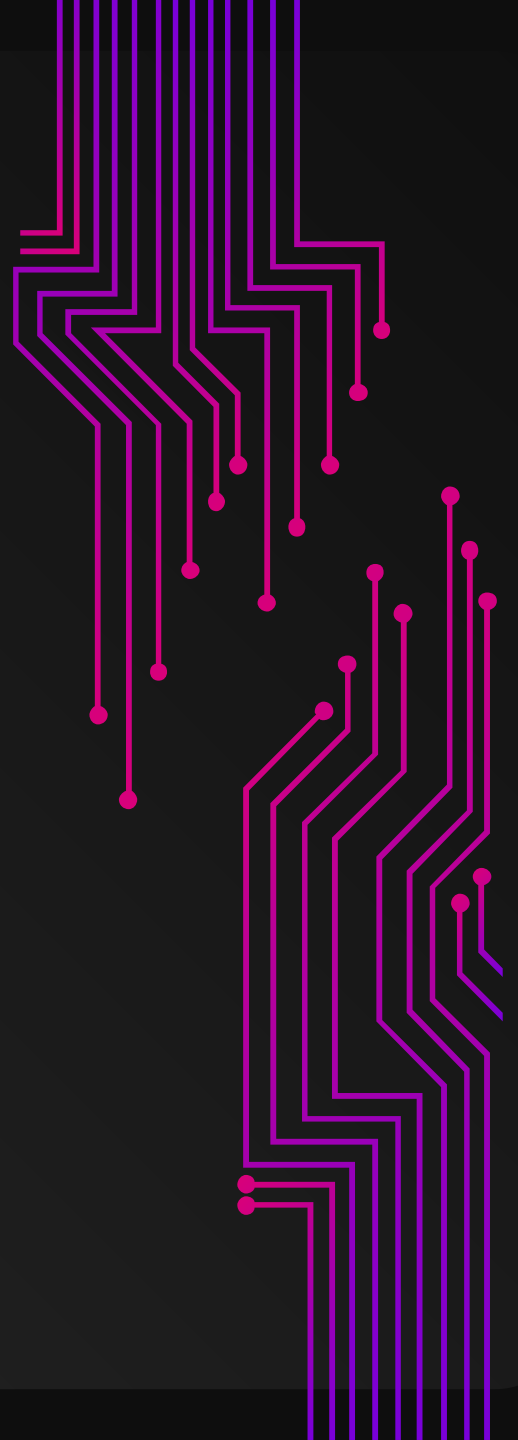
```
class Point {
  double x = 0;
  double y = 0;

  Point(double x, double y) {
    // See initializing formal parameters for a better way
    // to initialize instance variables.
    this.x = x;
    this.y = y;
  }
}
```

02

Advanced Classes

Methods
Getters and Setters
Objects





Methods

Methods in a class are functions that define the behavior of objects created from that class. You can call methods on objects to perform actions or computations. In the Person class, the greet method is an example of a method.

```
void sayHello() {
print('Hello, World!');
}

int add(int a, int b) {
    return a + b;
}
```

Getters and Setters

Dart allows you to define getters and setters to control access to class properties. Getters are used to retrieve property values, while setters are used to modify them. Here's an example...

```dart
class Rectangle {
  double _width;
  double _height;

  Rectangle(this._width, this._height);

  double get width => _width;
  set width(double value) {
    if (value > 0) {
      _width = value;
    }
  }

  double get area => _width * _height;
}

void main() {
  var rectangle = Rectangle(5, 3);
  print('Width: ${rectangle.width}'); // Output: Width: 5.0

  rectangle.width = 7;
  print('Width: ${rectangle.width}'); // Output: Width: 7.0

  print('Area: ${rectangle.area}'); // Output: Area: 21.0
}
```

Creating Objects

To create an object in Dart, you use the new keyword followed by the class constructor. Here's how to create two Person objects:

```
void main() {
var person1 = Person('Alice', 25);
var person2 = Person('Bob', 30);

person1.greet(); // Output: Hello, my name is Alice and I am 25 years old.
person2.greet(); // Output: Hello, my name is Bob and I am 30 years old.
}
```

Practise Time

Exercise 1: Creating a Class
Create a Dart class called Car with properties for make, model, and year. Add a constructor to initialize these properties. Then, create an object of the Car class and print its details.

Exercise 2: Bank Account Class
Define a BankAccount class with properties for accountNumber and balance. Include methods for deposit and withdraw. Create two objects from the class, deposit and withdraw funds, and print the account balances.

Exercise 3: Book Class
Create a Book class with properties for title, author, and publicationYear. Implement a method called printInfo that prints the book's information. Create two Book objects and call the printInfo method for each.

Practise Time

Exercise 4: Rectangle Class
Define a Rectangle class with properties for length and width. Add a method called calculateArea that returns the area of the rectangle. Create a Rectangle object and calculate its area.

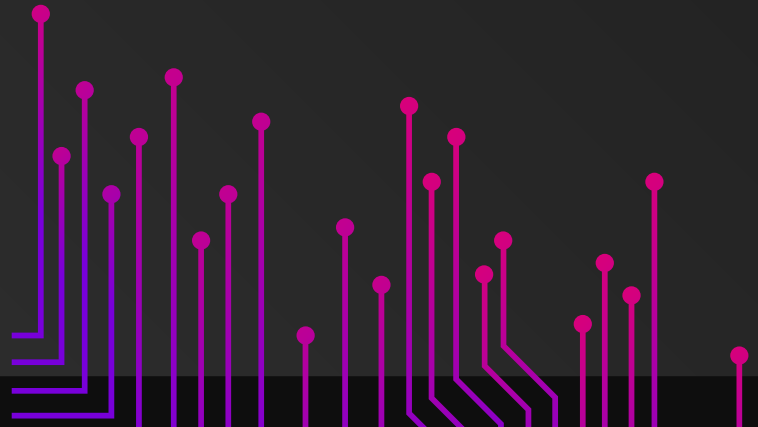
Exercise 5: Employee Class
Create an Employee class with properties for name, employeeID, and salary. Implement a method called giveRaise that increases the salary by a specified amount. Create an Employee object, give them a raise, and print their updated salary.

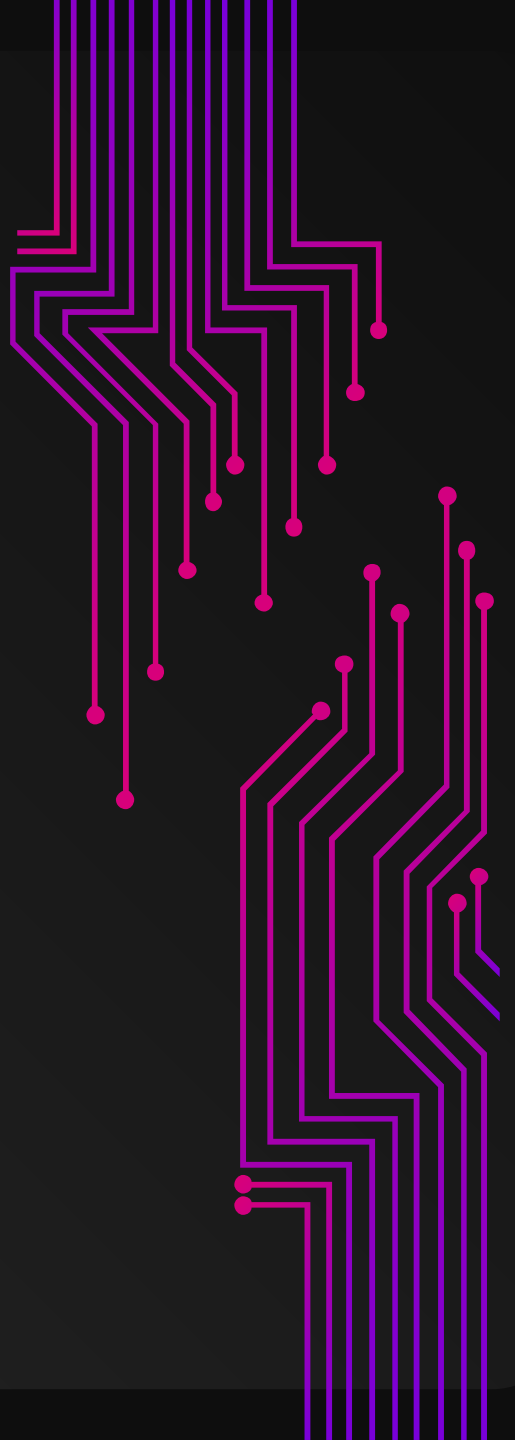
Exercise 6: Student Class
Define a Student class with properties for name, studentID, and a list of grades. Implement a method called calculateAverageGrade that calculates the average of the grades. Create a Student object, add grades, and calculate the average grade.

03

Basic OOP

Inheritance
Encapsulation
Polymorphism





Inheritance

Dart supports inheritance, allowing you to create a new class based on an existing class. The new class inherits properties and methods from the parent class and can override or extend them. Here's an example:

In this example, the Student class inherits from the Person class and overrides the greet method to include the major.

```
class Student extends Person {
  String major;

  Student(String name, int age, this.major) : super(name, age);

  @override
  void greet() {
    print('Hello, my name is $name, I am $age years old, and I am majoring in $major.');
  }
}

void main() {
  var student = Student('Eve', 21, 'Computer Science');
  student.greet(); // Output: Hello, my name is Eve, I am 21 years old, and I am majoring in Computer Science.
}
```

Encapsulation

Dart supports encapsulation by allowing you to mark class properties and methods as private using an underscore (_) prefix. Private properties can only be accessed within the same class, while private methods can only be called within the same class.

In this example, _accountNumber and _balance are private properties, and deposit, withdraw, and getBalance are methods for interacting with the bank account.

```
class BankAccount {
  String _accountNumber;
  double _balance;
  BankAccount(this._accountNumber, this._balance);

  void deposit(double amount) {
    if (amount > 0) {
      _balance += amount;
    }
  }
  void withdraw(double amount) {
    if (amount > 0 && amount <= _balance) {
      _balance -= amount;
    }
  }
  double getBalance() {
    return _balance;
  }
}
```

Polymorphism

Polymorphism is a fundamental concept in object-oriented programming that allows objects of different types to be treated as objects of a common type. Dart supports polymorphism through both compile-time polymorphism (static polymorphism) and runtime polymorphism (dynamic polymorphism).

Polymorphism

Compile-Time Polymorphism (Method Overloading):

In Dart, method overloading is achieved by declaring multiple methods with the same name but with different parameter lists. The Dart compiler determines which method to invoke based on the number and types of arguments provided at the call site.

```
class MathOperations {
  int add(int a, int b) {
    return a + b;
  }

  double add(double a, double b) {
    return a + b;
  }
}

void main() {
  MathOperations math = MathOperations();
  print(math.add(3, 4));       // Calls the int version of add
  print(math.add(2.5, 3.7));   // Calls the double version of add
}
```

In this example, the MathOperations class has two add methods—one for integers and another for doubles. The appropriate method is called based on the argument types provided at the call site

Polymorphism

Runtime Polymorphism (Method Overriding):

Dart supports runtime polymorphism through method overriding. Subclasses can provide a specific implementation for a method declared in a superclass.

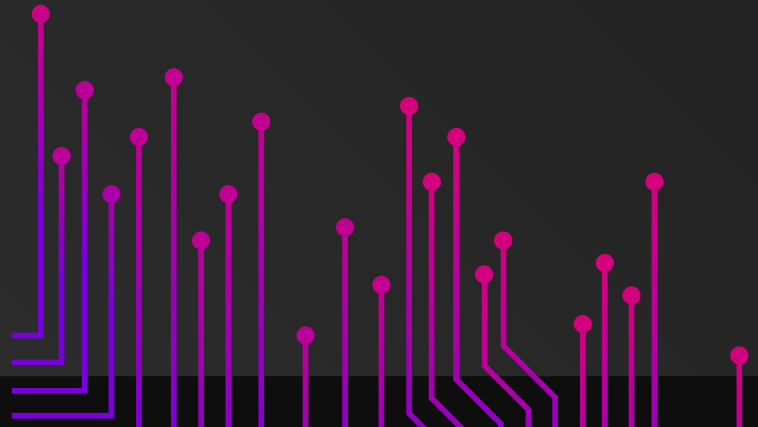
```
class Animal {
  void makeSound() {
    print("Some generic sound");
  }
}

class Cat extends Animal {
  @override
  void makeSound() {
    print("Meow");
  }
}

class Dog extends Animal {
  @override
  void makeSound() {
    print("Woof");
  }
}

void main() {
  Animal cat = Cat();
  Animal dog = Dog();

  cat.makeSound();  // Calls the overridden makeSound in Cat class
  dog.makeSound();  // Calls the overridden makeSound in Dog class
}
```

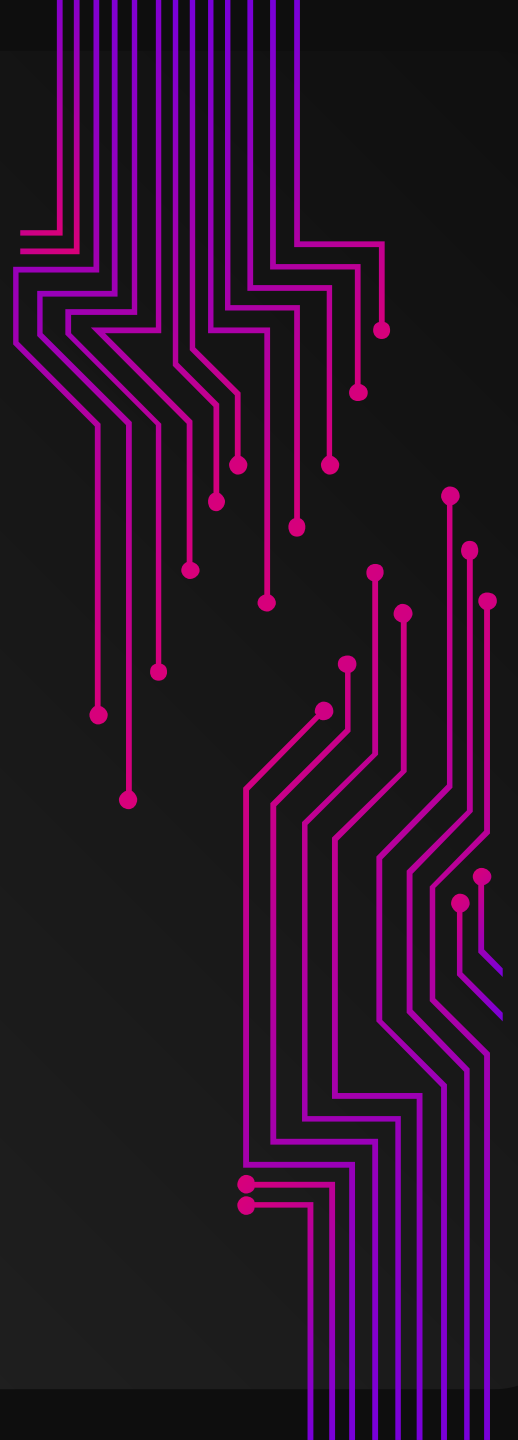
In this example, the Animal class has a makeSound method. The Cat and Dog classes extend Animal and override the makeSound method with specific implementations. The overridden method is called based on the actual type of the object at runtime.

04

More And More

Static Members
Factory Constructors
Class Extensions





Static Members

You can define static properties and methods in a class, which are associated with the class itself rather than instances of the class. Static members are accessed using the class name. For example:

In this code, pi is a static constant, and calculateCircleArea is a static method of the MathUtils class.

```
class MathUtils {
  static const double pi = 3.14159265359;

  static double calculateCircleArea(double radius) {
    return pi * radius * radius;
  }
}

void main() {
  print(MathUtils.calculateCircleArea(5)); // Output: 78.539816339745
}
```

Factory Constructors

Dart allows you to define factory constructors, which can return an instance of the class that may not necessarily be newly allocated. Factory constructors are often used for caching or returning a previously created object. For example:

In this code, the Singleton class ensures that only one instance of the class is created.

```
class Singleton {
  static Singleton? _instance;

  factory Singleton() {
    if (_instance == null) {
      _instance = Singleton._();
    }
    return _instance!;
  }

  Singleton._(); // Private constructor
}

void main() {
  var instance1 = Singleton();
  var instance2 = Singleton();

  print(identical(instance1, instance2)); // Output: true
}
```

Class Extensions

Dart allows you to extend existing classes with additional functionality using extensions. Extensions enable you to add methods to existing classes without modifying their source code. For example:

In this code, we create a StringUtilities extension to add an isPalindrome method to the String class.

```
extension StringUtilities on String {
  bool isPalindrome() {
    return this == this.split(”).reversed.join(”);
  }
}

void main() {
  var word = ’racecar’;
  print(word.isPalindrome()); // Output: true
}
```

Practise Time

Exercise 1: Shape Hierarchy
Create a hierarchy of shapes using inheritance. Define a base class called Shape with properties like color and methods like area. Then, create subclasses such as Circle and Rectangle that inherit from Shape and override the area method. Calculate and print the area of instances of these shapes.

Exercise 2: Bank Account Inheritance
Extend the BankAccount class from a previous exercise with inheritance. Create a SavingsAccount class that inherits from BankAccount and includes an additional property interestRate. Implement a method calculateInterest that calculates and adds interest to the account balance. Create a SavingsAccount object and demonstrate depositing, withdrawing, and calculating interest.

Practise Time

Exercise 3: Employee Inheritance
Build an Employee class hierarchy using inheritance. Define a base class called Employee with properties like name, employeeID, and salary. Then, create subclasses such as Manager and Developer that inherit from Employee and have additional properties and methods specific to their roles. Create instances of these subclasses and demonstrate their properties and methods.

Exercise 4: Vehicle Encapsulation
Create a Vehicle class with properties like make, model, and year. Use encapsulation by making these properties private and providing getters and setters to access and modify them. Ensure that the setter methods validate inputs (e.g., ensure the year is not in the future). Create a Vehicle object and demonstrate getting and setting its properties while adhering to encapsulation rules.

Questions !

Thank You !

Hassan Abdelkhalek